



INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Tocantins

Curso Superior de Tecnologia para Internet

Disciplina: Gerenciamento de Dados para Web **Professor:**

Fernando Jorge Ebrahim Lima e Silva

Aluno: Ulisses da Silva Bembem

Matrícula: 20222010350198

Exercícios de Múltipla Escolha (5 questões)

1 - Qual das alternativas a seguir NÃO é uma característica do AJAX?

- ☒ a) ~~É uma linguagem de programação.~~
- ☐ b) Permite atualizar uma página web sem recarregá-la.
- ☐ c) Utiliza o objeto XMLHttpRequest para acessar servidores web.
- ☐ d) Pode enviar dados para o servidor em segundo plano.

2 - Qual objeto é utilizado pelo AJAX para solicitar dados de um servidor web?

- ☒ a) ~~XMLHttpRequest~~
- ☐ b) ActiveXObject
- ☐ c) JavaScriptObject
- ☐ d) HTMLObject

3 - Qual método do objeto XMLHttpRequest é utilizado para enviar uma requisição para o servidor?

- ☐ a) open()
- ☒ b) ~~send()~~
- ☐ c) getRequestHeader()
- ☐ d) getAllResponseHeaders()

4 - Qual propriedade do objeto XMLHttpRequest armazena o status da requisição?

- ☐ a) responseText
- ☐ b) responseXML
- ☐ c) readyState

☒ ~~c) status~~

5 - Qual código de status HTTP indica que a requisição foi bem-sucedida?

- ☐ a) 403
- ☐ b) 404
- ☒ ~~c) 200~~
- ☐ d) 500

Exercícios Abertos (3 questões)

6 - Descreva o funcionamento do AJAX, detalhando os passos envolvidos na comunicação entre o navegador e o servidor.

- Criação do objeto XMLHttpRequest: O JavaScript cria um objeto `XMLHttpRequest` (ou alternativa para navegadores antigos) para gerenciar a requisição.
- Configuração da requisição: O método `open(method, url, async)` define o tipo de requisição (ex.: GET, POST), a URL do servidor e se é assíncrona (`true`).
- Definição de callback: A propriedade `onreadystatechange` é configurada com uma função que trata a resposta do servidor quando o estado da requisição muda.
- Envio da requisição: O método `send()` envia a requisição ao servidor. Para POST, dados podem ser passados como argumento.
- Processamento no servidor: O servidor recebe a requisição, processa e retorna uma resposta (ex.: JSON, texto, XML).
- Recebimento da resposta: O navegador detecta mudanças no `readyState`. Quando `readyState` é 4 (concluído) e `status` é 200 (sucesso), a resposta (em `responseText` ou `responseXML`) é processada.
- Atualização da interface: O JavaScript manipula a resposta para atualizar o DOM, exibindo dados na página sem recarregar.

7 - Compare e contraste os métodos HTTP GET e POST, indicando em quais situações cada um é mais apropriado para ser utilizado com AJAX.

GET:

- **Características:** Envia dados na URL (query string), é idempotente (múltiplas chamadas produzem o mesmo resultado), cacheável, limitado em tamanho (depende do navegador).
- **Vantagens:** Simples, rápido para requisições pequenas, suporta caching.
- **Desvantagens:** Dados visíveis na URL, menos seguro, limite de comprimento.

- **Uso com AJAX:** Ideal para recuperar dados (ex.: buscar lista de produtos, carregar JSON de uma API pública). Exemplo: `xhr.open("GET", "api/dados?user=123", true).`

POST:

- **Características:** Envia dados no corpo da requisição, não é idempotente, não cacheável, suporta maior volume de dados.
- **Vantagens:** Mais seguro (dados não aparecem na URL), adequado para dados grandes ou sensíveis.
- **Desvantagens:** Mais complexo, não cacheável, ligeiramente mais lento.
- **Uso com AJAX:** Ideal para enviar dados ao servidor (ex.: formulários, upload de arquivos, autenticação). Exemplo: `xhr.open("POST", "api/salvar", true)` com `xhr.send(dados).`

Quando usar:

- **GET:** Para requisições de leitura, como carregar dados de uma API ou buscar recursos.
- **POST:** Para ações que modificam o servidor, como criar, atualizar ou enviar dados sensíveis.

8 - Explique a importância da propriedade `onreadystatechange` do objeto `XMLHttpRequest` e como ela é utilizada para tratar a resposta do servidor.

A propriedade `onreadystatechange` do objeto `XMLHttpRequest` é essencial para monitorar mudanças no estado da requisição, permitindo tratar a resposta do servidor de forma assíncrona. Ela armazena uma função que é chamada toda vez que a propriedade `readyState` muda.

- **Importância:**
 - Permite verificar o progresso da requisição (ex.: enviada, recebendo dados, concluída).
 - Garante que a resposta seja processada apenas quando a requisição está completa (`readyState === 4`) e bem-sucedida (`status === 200`).
 - Evita erros ao acessar `responseText` ou `responseXML` antes da conclusão.
- **Como é utilizada:**
 - Define-se uma função para `onreadystatechange` que verifica `readyState` e `status`.
 - Exemplo:

- javascript

```
xhr.onreadystatechange = function () {  
    if (xhr.readyState === 4 && xhr.status === 200) {  
        console.log(xhr.responseText); // Processa a resposta  
    }  
};
```

- A função pode manipular o DOM, exibir erros (ex.: `if (xhr.status === 404)`) ou processar dados JSON.

Exercícios Práticos (2 questões)

9 - Código Cross-Browser: Escreva um trecho de código JavaScript que crie um objeto XMLHttpRequest de forma compatível com diferentes navegadores (incluindo versões antigas do Internet Explorer).

```
function criarXMLHttpRequest() {  
    let xhr = null;  
    try {  
        // Navegadores modernos (Chrome, Firefox, Safari, etc.)  
        xhr = new XMLHttpRequest();  
    } catch (e) {  
        try {  
            // Internet Explorer 6 e anteriores  
            xhr = new ActiveXObject("Msxml2.XMLHTTP");  
        } catch (e) {  
            try {  
                // Internet Explorer 5.5 e anteriores  
                xhr = new ActiveXObject("Microsoft.XMLHTTP");  
            } catch (e) {  
                console.error("Não foi possível criar o objeto XMLHttpRequest.");  
            }  
        }  
    }  
    return xhr;  
}
```

```
}
```

```
// Exemplo de uso
```

```
const xhr = criarXMLHttpRequest();
```

```
if (xhr) {
```

```
    console.log("Objeto XMLHttpRequest criado com sucesso!");
```

```
}
```

10 - Requisição GET com Tratamento de Resposta: Crie um código JavaScript que realize uma requisição GET para um arquivo chamado "dados.txt" e exiba o conteúdo desse arquivo em um elemento HTML com o id "resultado". Inclua o tratamento da propriedade onreadystatechange para garantir que a resposta seja exibida apenas quando a requisição for concluída com sucesso.

```
<!DOCTYPE html>
```

```
<html lang="pt-BR">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Requisição AJAX</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Conteúdo do Arquivo</h1>
```

```
    <div id="resultado">Aguardando dados...</div>
```

```
<script>
```

```
    // Criar o objeto XMLHttpRequest
```

```
    const xhr = new XMLHttpRequest();
```

```
    // Configurar a requisição GET
```

```
    xhr.open("GET", "dados.txt", true);
```

```
    // Definir o tratamento da resposta
```

```
    xhr.onreadystatechange = function () {
```

```
        if (xhr.readyState === 4) { // Requisição concluída
```

```
            if (xhr.status === 200) { // Sucesso
```

```
                document.getElementById("resultado").innerText = xhr.responseText;
```

```
            } else {
```

```
                document.getElementById("resultado").innerText =
```

```
                    "Erro: Não foi possível carregar o arquivo (Status: " + xhr.status + ")";
```

```
            }
```

```
        }
```

```
    };
```

```
    // Enviar a requisição
```

```
    xhr.send();
```

```
</script>  
</body>  
</html>
```